

1.3 Machine Learning Basics

These basics are meant as a recap, but if things are going too fast of if you haven't seen any machine learning lecture before, you can have a look into Goodfellow et al.: Deep Learning, Chapter 5 (<http://www.deeplearningbook.org/contents/ml.html>).

1.3.1 Learning Problems

Supervised learning

In supervised learning, model parameters are learned using a dataset of data-label pairs $\{(x_i, y_i)\}_{i=1}^N$. Here x is the input and y is the output and we have N such pairs. Examples include classification, regression and structured prediction problems.

Unsupervised learning

Model parameters are learned using a dataset without labels $\{x_i\}_{i=1}^N$, so just with inputs. Examples include clustering, dimensionality reduction and generative models.

Self-supervised learning

Model parameters are learned using a dataset of data-data pairs $\{(x_i, x'_i)\}_{i=1}^N$. Examples include self-supervised stereo and optical flow estimation and contrastive learning.

Reinforcement learning

Model parameters are learned using active exploration from sparse rewards instead of a fixed dataset. Examples

include deep q learning, gradient policy and actor critique.

In this lecture, we will cover the first three learning problems.

1.3.2 Supervised Learning

Classification / Regression:

In classification, the goal is to predict a discrete class label using a function f with input $\mathcal{X}$:

$$f : \mathcal{X} \rightarrow \mathbb{N}$$

In regression, we are interested in predicting from an arbitrary input $\mathcal{X}$ a continuous variable.

$$f : \mathcal{X} \rightarrow \mathbb{R}$$

Inputs $x \in \mathcal{X}$ can be **any kind of objects** images, text, a sequence of amino acids, ...

The output $y \in \mathbb{N}/y \in \mathbb{R}$ is either a **discrete or real number**, but only one single number. Examples are the label of an image as in classification, or regressing a single number for the stock market, a density estimation, ...

Structured Output Learning:

In structured output learning, both the input and the output are structured objects.

$$f : \mathcal{X} \rightarrow \mathcal{Y}$$

The inputs $x \in \mathcal{X}$ can be **any kind of objects** and the outputs $y \in \mathcal{Y}$ are **complex (structured) objects** such as images, text, parse trees, folds of a protein, computer programs, ...

Today we will focus on the regression problem. A model f always takes an input x and produces an output y . There are two main tasks, the learning task and the inference task.

Learning: Learning happens through estimating the parameters $\mathbf{w}$ from training data $\{(x_i, y_i)\}_{i=1}^N$

Inference: Given $\mathbf{w}$ from the learning task, novel predictions are made: $y = f_{\mathbf{w}}(x)$

Classification

An example is the mapping of an input image to the output label “Beach” or “No Beach”, thus performing the task of classification:

Mapping: $f_{\mathbf{w}} : \mathbb{R}^{W \times H} \rightarrow \{\text{“Beach”, “No Beach”}\}$

Regression

For a regression problem, a stock value prediction with N numbers as input and a single number as output are an example:

Mapping: $f_{\mathbf{w}} : \mathbb{R}^N \rightarrow \mathbb{R}$

Structured Prediction

An example for structured prediction problems is an audio signal as input and a sequence of words, forming a sentence as output.

Mapping: $f_{\mathbf{w}} : \mathbb{R}^N \rightarrow \{1, \dots, L\}^M$

Semantic segmentation is another example for structured prediction, where the input is an image and the output is a label map.

Mapping: $f_{\mathbf{w}} : \mathbb{R}^{W \times H} \rightarrow \{1, \dots, L\}^{W \times H}$

A third example of structured prediction is 3D reconstruction, the input is a set of images and the output is a 3D reconstruction. Mapping: $f_{\mathbf{w}} : \mathbb{R}^{W \times H \times N} \rightarrow \{0, 1\}^{M^3}$

For this mapping, suppose we want to predict 32^3 voxels, with a binary variable per voxel (occupied/free) $2^{32^3} = 2^{32768}$ different reconstructions could be predicted by the model. This number is even larger than the number of atoms in the universe, which is $\sim 2^{273}$.

1.3.3 Linear Regression

In linear regression, we want to make a prediction for the model parameters in a linear sense. Formally, this means: Let $\mathcal{X}$ denote a dataset of size N and let $(\mathbf{x}_i, y_i) \in \mathcal{X}$ denote its elements ($y_i \in \mathbb{R}$). The goal is, to predict y for a previously unseen input $\mathbf{x}$. The input $\mathbf{x}$ may be multidimensional.

As an example, we try to fit a line as the "ground truth" over some noisy observations, which are samples from the model with added noise. This is what we provide to the learning algorithm. When we execute that model at a novel location $\mathbf{x}$, we get a sensible response $\mathbf{y}$. This is done by defining an error function.

The **error function** $E(\mathbf{w})$ measures the displacement along the y dimension between the data points and the model $f(\mathbf{x}, \mathbf{w})$ specified by the parameters $\mathbf{w}$. It aims at minimizing the overall error.

$$\begin{aligned} f(\mathbf{x}, \mathbf{w}) &= \mathbf{w}^\top \mathbf{x} \\ E(\mathbf{w}) &= \sum_{i=1}^N (f(\mathbf{x}_i, \mathbf{w}) - y_i)^2 \\ &= \sum_{i=1}^N (\mathbf{x}_i^\top \mathbf{w} - y_i)^2 \\ &= \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 \end{aligned}$$

In linear regression, we have a function represented as a linear model. $\mathbf{w}^\top \mathbf{x}$ is the inner product of the weight vector $\mathbf{w}$ and the input features $\mathbf{x}$. The error function is the sum over all data points of the square of the difference in the $\mathbf{y}$ direction between the prediction of the model and the observation $\mathbf{y}$. This can be written as the squared ℓ_2 norm of the matrix $\mathbf{X}$ times the weight vector $\mathbf{w}$ minus the vector $\mathbf{y}$.

The **gradient of the error function** with respect to the parameters $\mathbf{w}$ is given by:

$$\begin{aligned} \nabla_{\mathbf{w}} E(\mathbf{w}) &= \nabla_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 \\ &= \nabla_{\mathbf{w}} (\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) \\ &= \nabla_{\mathbf{w}} (\mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2\mathbf{w}^\top \mathbf{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}) \\ &= 2\mathbf{X}^\top \mathbf{X} \mathbf{w} - 2\mathbf{X}^\top \mathbf{y} \end{aligned}$$

As $E(\mathbf{w})$ is quadratic and convex in $\mathbf{w}$, its minimizer (wrt. $\mathbf{w}$) is given in closed form. This means that we can set $\nabla_{\mathbf{w}} E(\mathbf{w})$ to zero to obtain a closed form solution:

$$\nabla_{\mathbf{w}} E(\mathbf{w}) = 0 \Rightarrow \mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

The matrix $(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$ is also called **Moore-Penrose inverse** or pseudoinverse because it allows us, to compute the inverse of a non-square matrix, which is the minimizer of this least-square problem.

Example: Line Fitting

A linear least squares fit of the model $f(\mathbf{x}, \mathbf{w}) = w_0 + w_1 x$ to the data points can be obtained with line fitting. The error function $E(\mathbf{w})$ wrt. parameter w_1 is a square function that produces a parabola. In order to minimize the error, we must find the lowest point of the parabola.

Example: Polynomial Curve Fitting

Let us choose a **polynomial of order M** to model dataset $\mathcal{X}$:

$$f(x, \mathbf{w}) = \sum_{j=0}^M w_j x^j = \mathbf{w}^\top \mathbf{x} \quad \text{with features} \quad \mathbf{x} = (1, x^1, x^2, \dots, x^M)^\top$$

This function is linear in $\mathbf{w}$, but it is not linear in x .

There are two tasks, the training and the inference. In training, we want to estimate $\mathbf{w}$ from dataset $\mathcal{X}$ and during inference we want to predict y for novel x given estimated $\mathbf{w}$. Note that these features can be anything, including multi-dimensional inputs (e.g., images, audio), radial basis functions, sine/cosine functions, etc. In this example of polynomial curve fitting we have monomials.

For estimating $\mathbf{w}$ from $\mathcal{X}$, we define a squared error function, e.g.:

$$E(\mathbf{w}) = \sum_{i=1}^N (f(x_i, \mathbf{w}) - y_i)^2$$

The goal is, to optimize error function wrt. the parameters $\mathbf{w}$.

The error function is linear in $\mathbf{w}$ but not in x . This means, that wrt. $\mathbf{w}$ we obtain the same problem as before; we have a quadratic problem with a closed form solution:

$$E(\mathbf{w}) = \sum_{i=1}^N (f(x_i, \mathbf{w}) - y_i)^2 = \sum_{i=1}^N (\mathbf{w}^\top \mathbf{x}_i - y_i)^2 = \sum_{i=1}^N \left(\sum_{j=0}^M w_j x_i^j - y_i \right)^2$$

It can be rewritten in the **matrix-vector notation** (i.e., as linear regression problem).

$$E(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

with feature matrix $\mathbf{X}$, observation vector $\mathbf{y}$ and weight vector $\mathbf{w}$:

$$\mathbf{X} = \begin{pmatrix} \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_i & x_i^2 & \dots & x_i^M \\ \vdots & \vdots & \vdots & & \vdots \end{pmatrix} \quad \mathbf{y} = \begin{pmatrix} \vdots \\ y_i \\ \vdots \end{pmatrix} \quad \mathbf{w} = \begin{pmatrix} w_0 \\ \vdots \\ w_M \end{pmatrix}$$

Example for Polynomial Curve Fitting Results

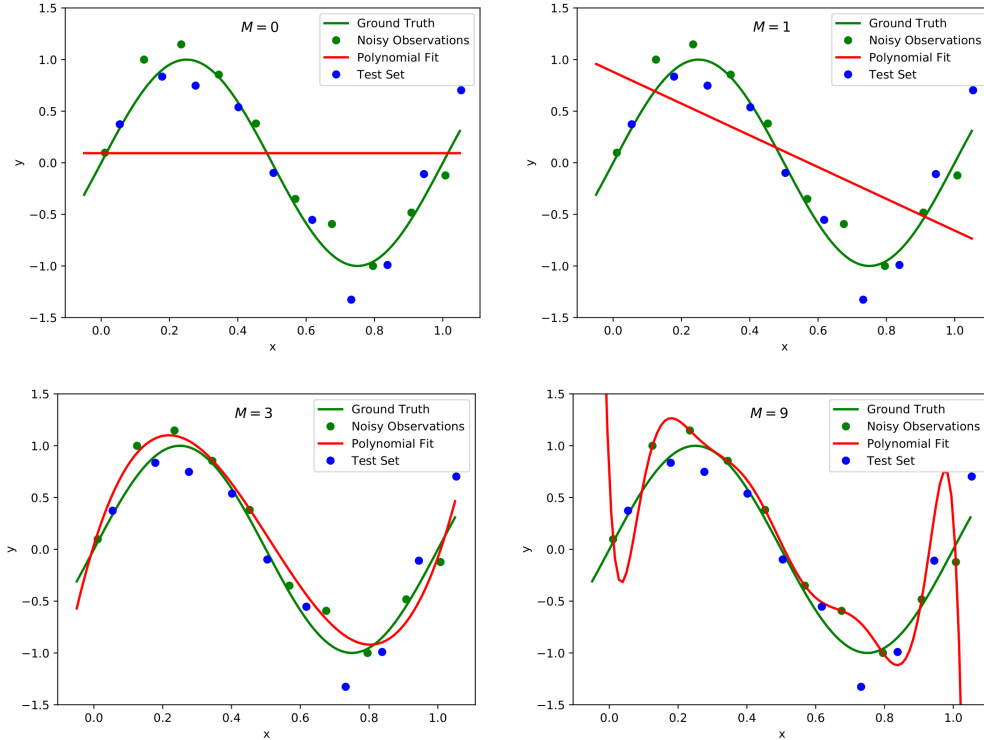


Figure 1: **Polynomial Curve fitting.** Plots of polynomials of various degrees M (red) fitted to the data (green). We observe underfitting ($M = 0/1$) and overfitting ($M = 9$). Choosing the best fitting degree M is a model selection problem.

Capacity, Overfitting and Underfitting

In summary, the goal of polynomial curve fitting is to perform well on new, previously unseen inputs (test set, blue in Fig. 1), not only on the training set (green in Fig. 1). This is called **generalization** and separates

machine learning from optimization. In optimization, we are just interested in fitting a model to observations while in machine learning, we are always interested in a model that generalizes well. The assumption that is often made in statistical learning theory, is that the training and test data are independent and identically (i.i.d.) drawn from the data distribution $p_{data}(x, y)$. This is important, because this assures that we can make certain statements about the learning problem. In the case of Fig. 1 the data distribution has been chosen as follows:

$$p_{data}(x) = \mathcal{U}(0, 1)$$

$$p_{data}(y|x) = \mathcal{N}(\sin(2\pi x), \sigma)$$

We have an underlying sine curve with x -locations between 0 and 1. On this basis we sample a y -location by adding some gaussian noise with the standard deviation σ to the sine curve.

Clarification of the used terminology:

Capacity refers to the complexity of functions which can be represented by the model f . In Fig. 1, for $M = 0/1$ the capacity is too low, for $M = 3$ the capacity is about right, and for $M = 9$ the capacity is too high.

Underfitting refers to models being too simple, so they don't achieve low error, not even on the training set.

Overfitting refers to a scenario where the training error is very small, but test error (= generalization error) is large. So the model overfits the training data and doesn't generalize well to the test set. We want neither an underfitting nor an overfitting model, but a model that performs well on the test set and thus generalizes well. As in the example of the generalization error for various polynomial degrees M , the model selection problem is as follows: we want to select the model with the smallest generalization error.

In general, we split the dataset into a training, a validation and a test set in order to choose the best model. We choose hyperparameters (e.g., degree of polynomial, learning rate in neural net, ..) using the validation set. It is important to evaluate only once on test set (true labels are typically not available). When the dataset is small, use (k-fold) cross validation instead of a fixed split.

1.3.4 Ridge Regression

Ridge Regression is another way of reducing the model complexity, but not by discretely changing the degree or the order of the polynomial, but by adding a regularization term. The error function of the ridge regression problem looks as follows:

$$E(\mathbf{w}) = \sum_{i=1}^N (f(x_i, \mathbf{w}) - y_i)^2 + \lambda \sum_{j=0}^M w^2$$

$$= \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_2^2$$

The idea is, to discourage large parameters by adding a regularization term with strength λ . Since this problem is quadratic in $\mathbf{w}$, it also has a closed form solution: $\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$.

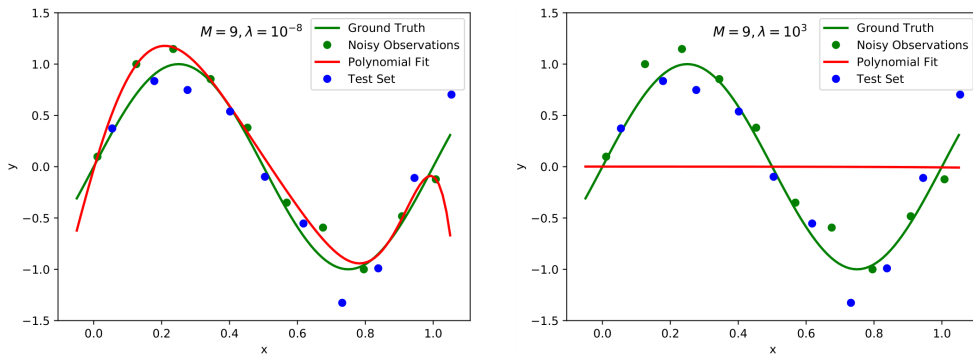


Figure 2: **Ridge Regression.** Plots of polynomial with degree $M = 9$ fitted to 10 data points using ridge regression. Left: weak regularization ($\lambda = 10^{-8}$). Right: strong regularization (right, $\lambda = 10^3$).

In Fig. 2 we can see, that for the formerly overfitting model with $M = 9$ we can obtain a much better fit with

a mild regularizer. With a strong regularizer the model remains constantly 0, which refers to the underfitting situation. If we choose a very small regularizer, parameters or model weights can become very large and the problem becomes ill-conditioned. If we increase the regularizer, the weights become smaller. Again, we want to select a model with the smallest generalization error on the validation set.

1.3.5 Estimators, Bias and Variance

Point Estimator:

A point estimator $g(\cdot)$ is a function that maps a dataset $\mathcal{X}$ to model parameters $\hat{\mathbf{w}}$: $\hat{\mathbf{w}} = g(\mathcal{X})$. We denote $\hat{\mathbf{w}}$ with a hat to explicitly mark that this is an estimate of the point estimator. An example is the estimator of the ridge regression model: $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$. A good estimator is a function that returns a parameter set close to the true one. We assume that the data $\mathcal{X} = \{(x_i, y_i)\}$ is drawn from a random process $(x_i, y_i) \sim p_{data}(\cdot)$, thus, any function of the data is random and $\hat{\mathbf{w}}$ is a random variable. This leads us to the terminology of bias and variance and the so-called bias and variance dilemma.

Properties of Point Estimators:

The bias of a point estimator is the expected value of the point estimate over all data sets that can be drawn from the data distribution minus the true value of the parameters:

$$\text{Bias}(\hat{\mathbf{w}}) = \mathbb{E}(\hat{\mathbf{w}}) - \mathbf{w}$$

This is how, in expectation, the point estimator deviates from the true parameters. This expectation is over all possible datasets $\mathcal{X}$. $\hat{\mathbf{w}}$ is unbiased $\Leftrightarrow \text{Bias}(\hat{\mathbf{w}}) = 0$. A good estimator has little bias.

The variance of a point estimator is the variance over $\hat{\mathbf{w}}$ over all datasets $\mathcal{X}$:

$$\text{Var}(\hat{\mathbf{w}}) = \mathbb{E}(\hat{\mathbf{w}}^2) - \mathbb{E}(\hat{\mathbf{w}})^2$$

The square root of the variance $\sqrt{\text{Var}(\hat{\mathbf{w}})}$ is called “standard error”. A good estimator also has low variance.

Bias-Variance Dilemma:

Statistical learning theory tells us that we can’t have a little bias as well as a low variance. There is a trade-off that we have to make.

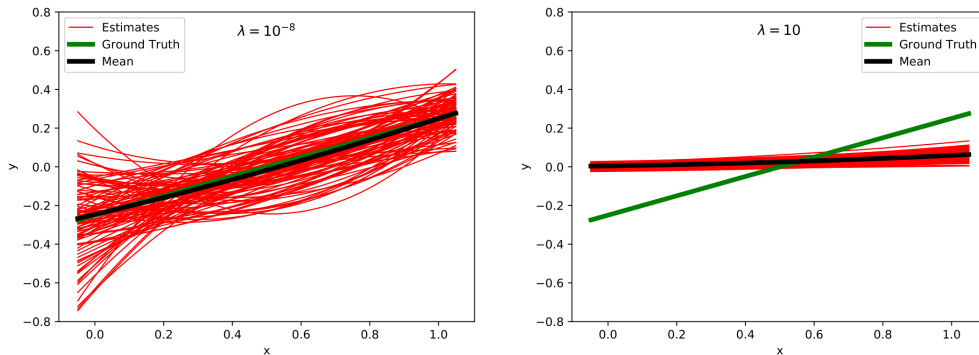


Figure 3: **Bias-Variance Dilemma.** Ridge regression with weak ($\lambda = 10^{-8}$) and strong ($\lambda = 10$) regularization. Green: True model. Black: Plot of model with mean parameters $\bar{\mathbf{w}} = \mathbb{E}(\mathbf{w})$. Red: Estimates, with high variance at the right and low variance at the left.

In Fig. 3 we can see that if we have a weak regularization, we have a large variance but the mean is a good fit to the ground truth, so the bias is small. With a strong regularization, we obtain less variance in the predictions, however, the model has a strong bias. The mean of the parameters (the red estimates) deviates strongly from the ground truth. So there is a bias-variance tradeoff which can be expressed mathematically: $\mathbb{E}[(\hat{\mathbf{w}} - \mathbf{w})^2] = \text{Bias}(\hat{\mathbf{w}})^2 + \text{Var}(\hat{\mathbf{w}})$. But we might not have to deal with this in all situations. E.g. in deep neural networks the test error decreases with network width. For further information see the blogpost at <https://www.bradyn Neal.com/bias-variance-tradeoff-textbooks-update>.

1.3.6 Maximum Likelihood Estimation

We now reinterpret our results by taking a probabilistic viewpoint. Let $\mathcal{X} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ be a dataset with samples drawn i.i.d. from the data distribution p_{data} . And let the model $p_{model}(y|\mathbf{x}, \mathbf{w})$ be a parametric family of probability distributions. Then the conditional maximum likelihood estimator for $\mathbf{w}$ is given by

$$\begin{aligned}\hat{\mathbf{w}}_{ML} &= \underset{\mathbf{w}}{\operatorname{argmax}} p_{model}(\mathbf{y}|\mathbf{X}, \mathbf{w}) \\ &\stackrel{\text{iid}}{=} \underset{\mathbf{w}}{\operatorname{argmax}} \prod_{i=1}^N p_{model}(y_i|\mathbf{x}_i, \mathbf{w}) \\ &= \underset{\mathbf{w}}{\operatorname{argmax}} \underbrace{\sum_{i=1}^N \log p_{model}(y_i|\mathbf{x}_i, \mathbf{w})}_{\text{Log-Likelihood}}\end{aligned}$$

Example: If we assume that the model distribution is a Gaussian, where the mean is a linear function, and the variance is σ : $p_{model}(y|\mathbf{x}, \mathbf{w}) = \mathcal{N}(y|\mathbf{w}^\top \mathbf{x}, \sigma)$, we obtain

$$\begin{aligned}\hat{\mathbf{w}}_{ML} &= \underset{\mathbf{w}}{\operatorname{argmax}} \sum_{i=1}^N \log p_{model}(y_i|\mathbf{x}_i, \mathbf{w}) \\ &= \underset{\mathbf{w}}{\operatorname{argmax}} \sum_{i=1}^N \log \left[\frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2\sigma^2}(\mathbf{w}^\top \mathbf{x}_i - y_i)^2} \right] \\ &= \underset{\mathbf{w}}{\operatorname{argmax}} - \sum_{i=1}^N \frac{1}{2} \log(2\pi\sigma^2) - \sum_{i=1}^N \frac{1}{2\sigma^2} (\mathbf{w}^\top \mathbf{x}_i - y_i)^2 \\ &= \underset{\mathbf{w}}{\operatorname{argmax}} - \sum_{i=1}^N (\mathbf{w}^\top \mathbf{x}_i - y_i)^2 \\ &= \underset{\mathbf{w}}{\operatorname{argmin}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2\end{aligned}$$

We see that choosing $p_{model}(y|\mathbf{x}, \mathbf{w})$ to be Gaussian causes maximum likelihood to yield exactly the same least squares estimator derived before:

$$\hat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmin}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

There are various variations possible here. If we were choosing e.g. the model distribution $p_{model}(y|\mathbf{x}, \mathbf{w})$ as a Laplace distribution, we would obtain an estimator that minimizes the ℓ_1 norm: $\hat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmin}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_1$. Or otherwise assuming a Gaussian distribution over the parameters $\mathbf{w}$ and performing a maximum a-posteriori (MAP) estimation yields to ridge regression: $\underset{\mathbf{w}}{\operatorname{argmax}} p(\mathbf{w}|\mathbf{y}, \mathbf{X}) = \underset{\mathbf{w}}{\operatorname{argmax}} p(\mathbf{y}|\mathbf{X}, \mathbf{w})p(\mathbf{w})$.

This shows that there is a connection between maximum likelihood estimation and the linear least squares problems that have been explained earlier. This is important, because maximum likelihood estimators are a very important tool in statistical learning theory because they are known to be consistent under mild assumptions. This means, that as the number of training samples approaches infinity $N \rightarrow \infty$, the maximum likelihood (ML) estimate converges to the true parameters. They are also very efficient: The ML estimate converges most quickly as N increases. These theoretical considerations make ML estimators appealing.